

More Advanced Solidity and Design Patterns

Peter Robinson, January 12, 2022

This talk is a sequel to my previous talks,

- "Advanced Solidity and Design Patterns" (2018),
- "EVM: From Solidity to byte code, memory and storage" (2020) and
- Solidity v0.6.x (2020).

Please watch the EVM talk before attending this talk:

https://www.youtube.com/watch?v=RxL_1AfV7N4

What is *Advanced* Solidity?

What was Advanced Solidity back in 2018?

Advanced Solidity and Design Patterns

Peter Robinson
July 25, 2018



<https://www.youtube.com/watch?v=VhzafmGGmzo&t=10s>

2018 Advanced Solidity and Design Patterns

- Upgrade pattern using a data holder contract.
- Touched on assembler.
- Lots of small stuff.

=> All still relevant

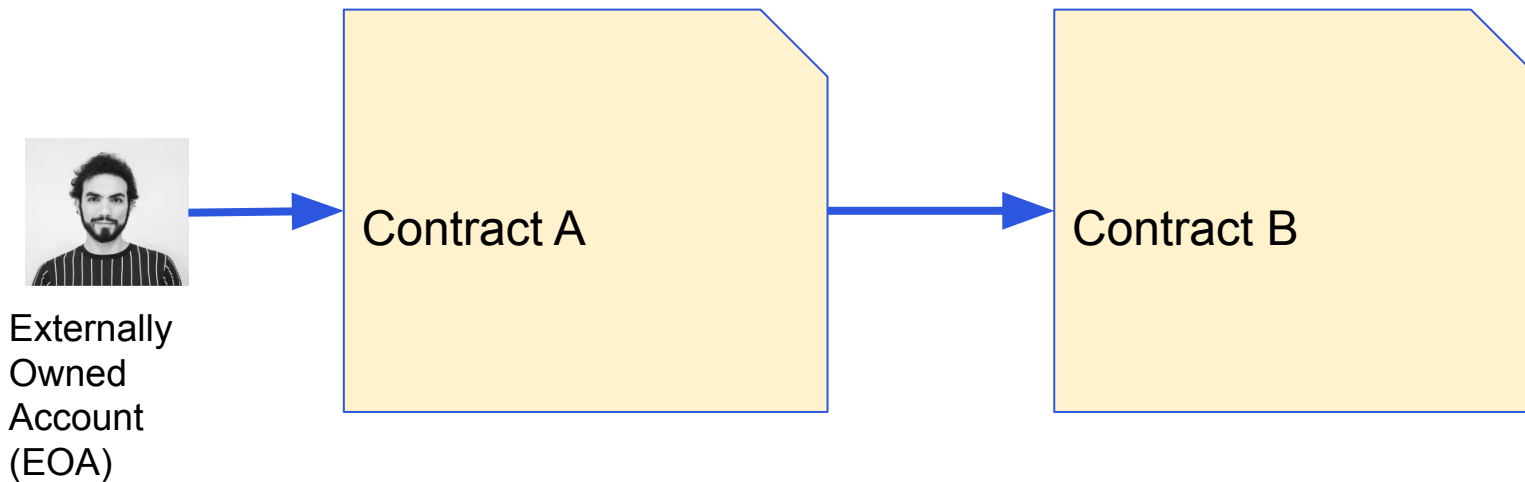
What is *Advanced* Solidity?

Agenda

- Cross-contract calls.
- Upgrade pattern using Open Zeppelin's upgradable contract.
- Hidden parameters.

Cross-Contract Calls

Cross-Contract Calls



Call



Externally
Owned
Account
(EOA)



Contract A

`msg.sender == EOA`
Context: Contract A



Contract B

`msg.sender == Contract A`
Context: Contract B

Call Code



Externally
Owned
Account
(EOA)



Contract A

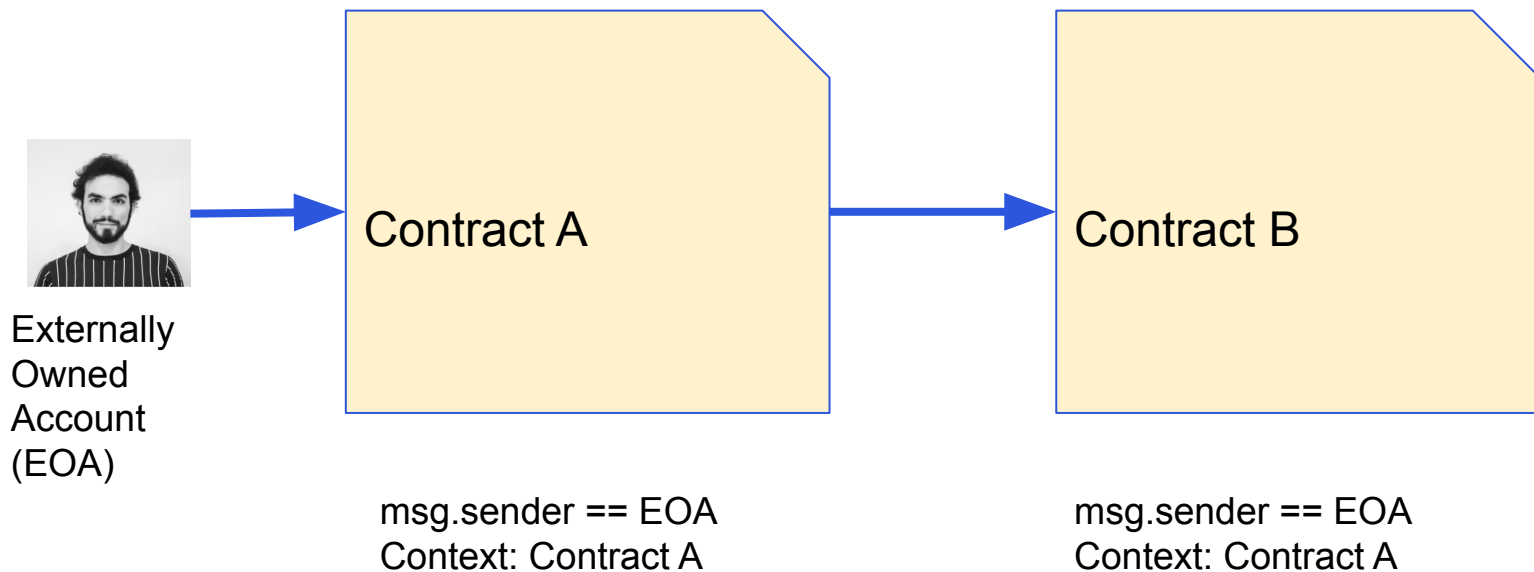
`msg.sender == EOA`
Context: Contract A



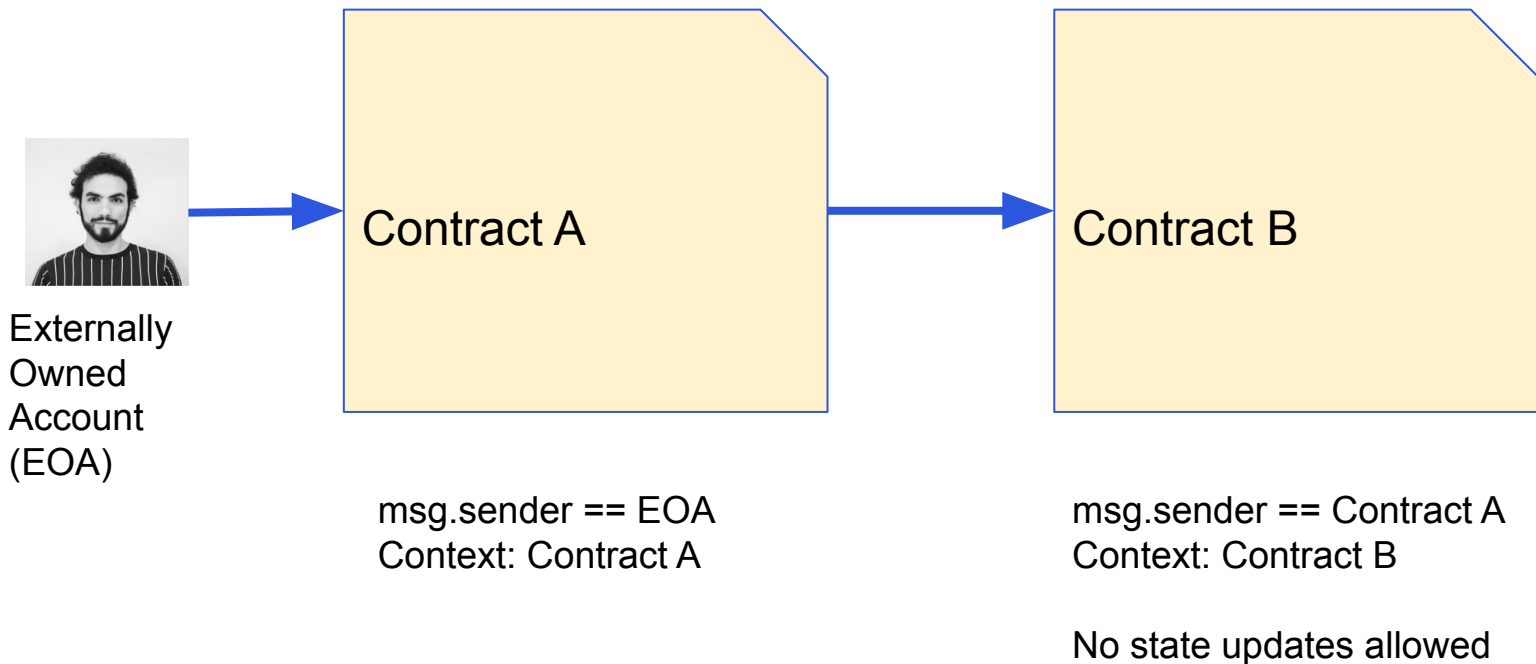
Contract B

`msg.sender == Contract A`
Context: Contract A

Delegate Call



Static Call



Memory Layout

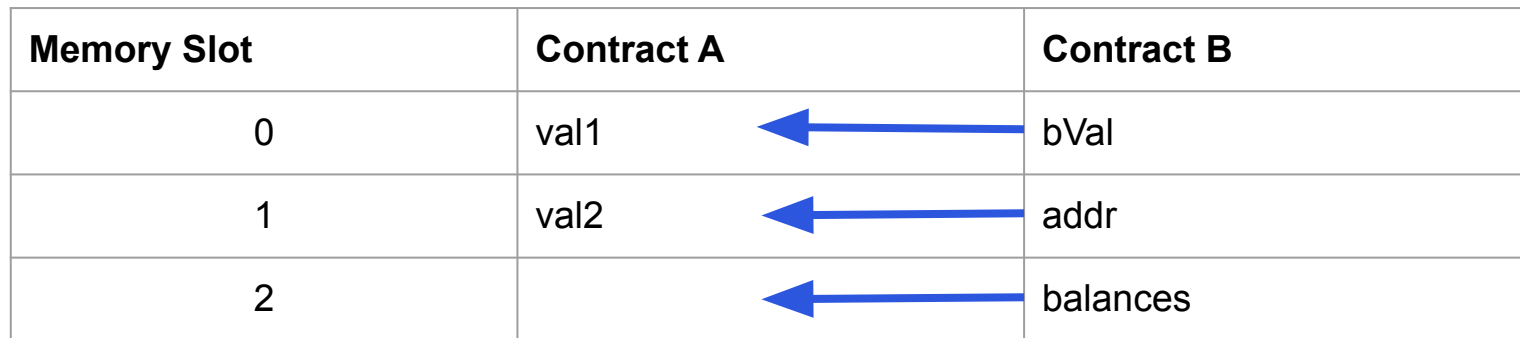
```
contract ContractA {  
    uint256 public val1;  
    uint256 public val2;  
}  
contract ContractB {  
    uint256 public bVal;  
    address public addr;  
    mapping(address => uint256) public balances;  
}
```

Memory Layout

Memory Slot	Contract A	Contract B
0	val1	bVal
1	val2	addr
2		balances

Memory Layout: Delegate Call

Memory Slot	Contract A	Contract B
0	val1	bVal
1	val2	addr
2		balances



For Delegate Call and Call Code, Contract B executes in the context of Contract A.

Call Types

Call Type	msg.sender in Contract B	Contract B's context	Contract B can update state	Usage
call	Contract A	Contract B	yes	Calls that update state
call code	Contract A	Contract A	yes	Deprecated
delegate call	Contract A's msg.sender	Contract A	yes	Replaces “call code”. Linked Libraries.
static call	Contract A	Contract B	no	View calls

Cross-Contract Calls

More Detail

Cross-Contract Calls

```
contract ContractA {
    ContractB immutable public conB;
    uint256 public val;
    constructor (address _conB) {
        conB = ContractB(_conB);
    }
    function callStuff1(bool _fail) external {
        val = conB.stuff1(_fail);
    }
}
```

```
contract ContractB {
    uint256 public bVal;

    function stuff1(bool _fail) external returns (uint256) {
        require(!_fail, "Failed");
        bVal = 2;
        return 3;
    }
}
```

Cross-Contract Calls

```
contract ContractA {  
    ContractB immutable public conB;  
    uint256 public val;  
    constructor (address _conB) {  
        conB = ContractB(_conB);  
    }  
    function callStuff1(bool _fail) external {  
        val = conB.stuff1(_fail);  
    }  
}
```

```
contract ContractB {  
    uint256 public bVal;  
  
    function stuff1(bool _fail) external returns (uint256) {  
        require(!_fail, "Failed");  
        bVal = 2;  
        return 3;  
    }  
}
```

Byte Code

PC: 0x117, opcode: JUMPDEST

PC: 0x118, opcode: POP

PC: 0x119, opcode: GAS

PC: 0x11a, opcode: CALL

PC: 0x11b, opcode: ISZERO

PC: 0x11c, opcode: DUP1

PC: 0x11d, opcode: ISZERO

PC: 0x11e, opcode: PUSH2 0x012b

PC: 0x121, opcode: JUMPI

PC: 0x122, opcode: RETURNDATASIZE

PC: 0x123, opcode: PUSH1 0x00

PC: 0x125, opcode: DUP1

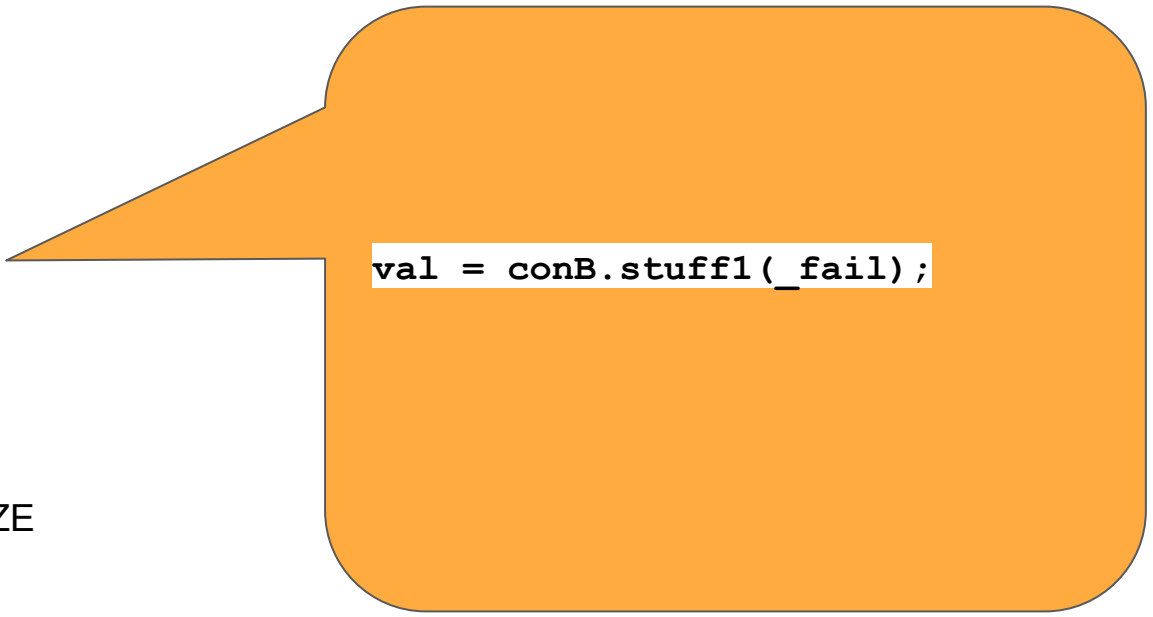
PC: 0x126, opcode: RETURNDATACOPY

PC: 0x127, opcode: RETURNDATASIZE

PC: 0x128, opcode: PUSH1 0x0

PC: 0x12a, opcode: REVERT

....



```
val = conB.stuff1(_fail);
```

Byte Code

PC: 0x117, opcode: JUMPDEST

PC: 0x118, opcode: POP

PC: 0x119, opcode: GAS

PC: 0x11a, opcode: CALL

PC: 0x11b, opcode: ISZERO

PC: 0x11c, opcode: DUP1

PC: 0x11d, opcode: ISZERO

PC: 0x11e, opcode: PUSH2 0x012b

PC: 0x121, opcode: JUMPI

PC: 0x122, opcode: RETURNDATASIZE

PC: 0x123, opcode: PUSH1 0x00

PC: 0x125, opcode: DUP1

PC: 0x126, opcode: RETURNDATACOPY

PC: 0x127, opcode: RETURNDATASIZE

PC: 0x128, opcode: PUSH1 0x0

PC: 0x12a, opcode: REVERT

....

```
val = conB.stuff1(_fail);
```

If the call returned 0, then an error has occurred.

Byte Code

PC: 0x117, opcode: JUMPDEST

PC: 0x118, opcode: POP

PC: 0x119, opcode: GAS

PC: 0x11a, opcode: CALL

PC: 0x11b, opcode: ISZERO

PC: 0x11c, opcode: DUP1

PC: 0x11d, opcode: ISZERO

PC: 0x11e, opcode: PUSH2 0x012b

PC: 0x121, opcode: JUMPI

PC: 0x122, opcode: RETURNDATASIZE

PC: 0x123, opcode: PUSH1 0x00

PC: 0x125, opcode: DUP1

PC: 0x126, opcode: RETURNDATACOPY

PC: 0x127, opcode: RETURNDATASIZE

PC: 0x128, opcode: PUSH1 0x0

PC: 0x12a, opcode: REVERT

....

```
val = conB.stuff1(_fail);
```

result = "the call returned 0"

if (result == 0) jump to 0x012b and fetch val

Byte Code

PC: 0x117, opcode: JUMPDEST
PC: 0x118, opcode: POP
PC: 0x119, opcode: GAS
PC: 0x11a, opcode: CALL
PC: 0x11b, opcode: ISZERO
PC: 0x11c, opcode: DUP1
PC: 0x11d, opcode: ISZERO
PC: 0x11e, opcode: PUSH2 0x012b
PC: 0x121, opcode: JUMPI

PC: 0x122, opcode: RETURNDATASIZE
PC: 0x123, opcode: PUSH1 0x00
PC: 0x125, opcode: DUP1
PC: 0x126, opcode: RETURNDATACOPY
PC: 0x127, opcode: RETURNDATASIZE
PC: 0x128, opcode: PUSH1 0x0
PC: 0x12a, opcode: REVERT

....

```
val = conB.stuff1(_fail);
```

otherwise forward the revert message.

The revert message from ContractB is in the return data.

Copy the data returned from Contract B to memory. Push onto the stack the return data size and location of the data.

Coding that into Solidity

```
function callStuff2(bool _fail) external {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector.
            returnValueEncoded := add(returnValueEncoded, 0x04)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Coding that into Solidity

Create data to be sent to the function:
bytes4 function selector
function parameters

```
function callStuff2(bool _fail) external {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector.
            returnValueEncoded := add(returnValueEncoded, 0x04)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Coding that into Solidity

Call the function on the contract

```
function callStuff2(bool _fail) external {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector.
            returnValueEncoded := add(returnValueEncoded, 0x04)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Coding that into Solidity

Returns (bool, bytes)

```
function callStuff2(bool _fail) external returns (bool, bytes) {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector..
            returnValueEncoded := add(returnValueEncoded, 0x04)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Coding that into Solidity

If success, then the return value encoded is the function return value

```
function callStuff2(bool _fail) external  
    bool success;  
    bytes memory returnValueEncoded;  
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);  
    (success, returnValueEncoded) = address(conB).call(funcParams);  
    if (success) {  
        val = abi.decode(returnValueEncoded, (uint256));  
    }  
    else {  
        // Assume a revert (and not a panic or user defined function)  
        assembly {  
            // Remove the function selector.  
            returnValueEncoded := add(returnValueEncoded, 0x04)  
        }  
        string memory revertReason = abi.decode(returnValueEncoded, (string));  
        revert(revertReason);  
    }  
}
```

Coding that into Solidity

Otherwise, the return value encoded is a panic value or revert message.

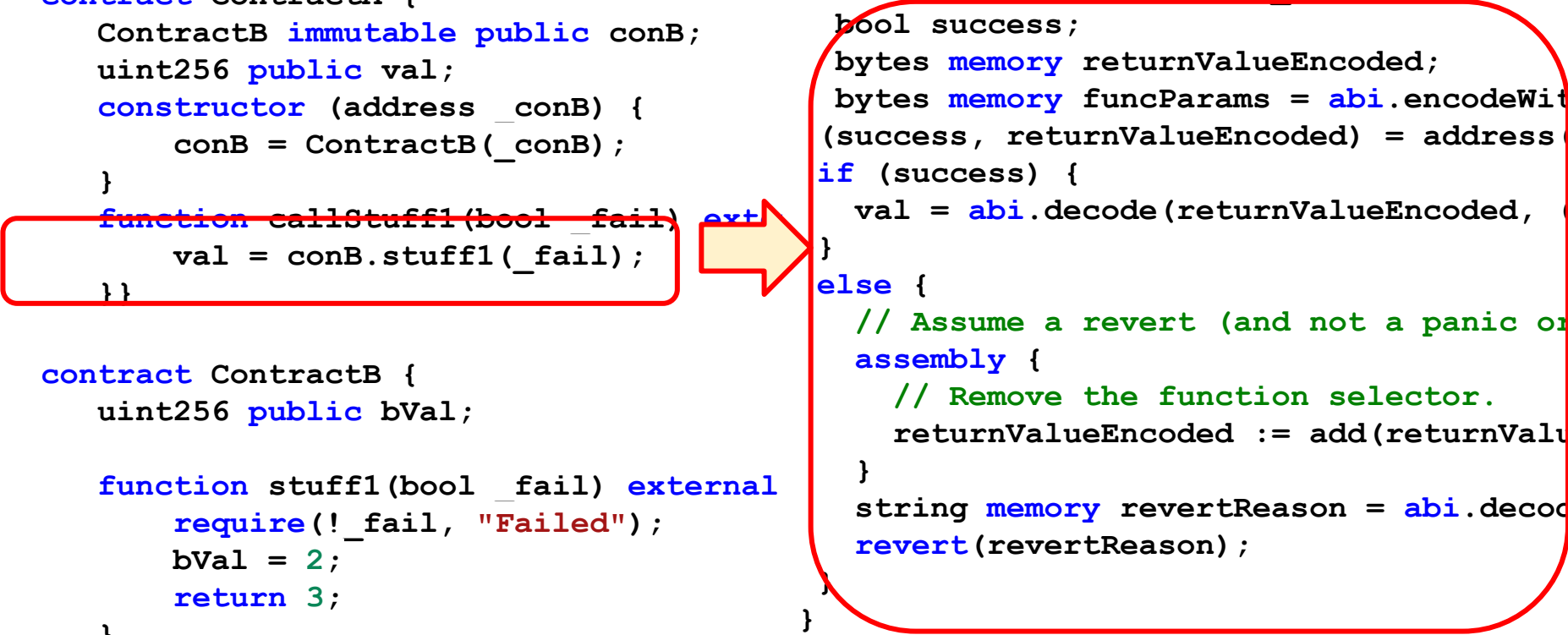
```
function callStuff2(bool _fail) external {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector.
            returnValueEncoded := add(returnValueEncoded, 0x04)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Why?

```
contract ContractA {
    ContractB immutable public conB;
    uint256 public val;
    constructor (address _conB) {
        conB = ContractB(_conB);
    }
    function callStuff1(bool _fail) external
        val = conB.stuff1(_fail);
}

contract ContractB {
    uint256 public bVal;

    function stuff1(bool _fail) external
        require(!_fail, "Failed");
        bVal = 2;
        return 3;
}
}
```



```
function callStuff2(bool _fail) external
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSignature(
        (success, returnValueEncoded) = address(conB).call(
        if (success) {
            val = abi.decode(returnValueEncoded, (uint256));
        }
        else {
            // Assume a revert (and not a panic or error)
            assembly {
                // Remove the function selector.
                returnValueEncoded := add(returnValueEncoded, 4)
            }
            string memory revertReason = abi.decode(returnValueEncoded, (string));
            revert(revertReason);
        }
    )
}
```

Why?

```
function callStuff2(bool _fail) external {
    bool success;
    bytes memory returnValueEncoded;
    bytes memory funcParams = abi.encodeWithSelector(conB.stuff1.selector, _fail);
    (success, returnValueEncoded) = address(conB).call(funcParams);
    if (success) {
        val = abi.decode(returnValueEncoded, (uint256));
    }
    else {
        // Assume a revert (and not a panic or user defined function)
        assembly {
            // Remove the function selector.
            returnValueEncoded := add(returnValueEncoded, 0x00)
        }
        string memory revertReason = abi.decode(returnValueEncoded, (string));
        revert(revertReason);
    }
}
```

Could use: delegateCall

Could do other interesting things...

Cross-Contract Calls Errors

Reverts and Panics

Error returns are in the format:

- bytes4 function selector
- ABI encoded parameters as defined by the function selector.

Panic and Revert are defined:

- **Panic:** `"0x4e487b71" = keccak256("Panic(uint256)")`
- **Revert:** `"0x08c379a0" = keccak256("Error(string)")`

For example code see: <https://github.com/ConsenSys/gpact/blob/main/common/common/src/main/solidity/ResponseProcessUtil.sol>

Panics

- 0x01: If you call `assert` with an argument that evaluates to false.
- 0x11: If an arithmetic operation results in underflow or overflow outside of an `unchecked { ... }` block.
- 0x12: If you divide or modulo by zero (e.g. `5 / 0` or `23 % 0`).
- 0x21: If you convert a value that is too big or negative into an enum type.
- 0x22: If you access a storage byte array that is incorrectly encoded.
- 0x31: If you call `.pop()` on an empty array.
- 0x32: If you access an array, `bytesN` or an array slice at an out-of-bounds or negative index (i.e. `x[i]` where `i >= x.length` or `i < 0`).
- 0x41: If you allocate too much memory or create an array that is too large.
- 0x51: If you call a zero-initialized variable of internal function type.

Custom Errors

```
error MyError(uint256 _errorCode, uint256 _balance);

function throwMyError(bool _fail) external {
    if (_fail) {
        revert MyError(23, bal);
    }
    val = 29;
}
```

Try / Catch

```
function doStuff4(bool _fail) external {
    try conB.stuff1(_fail) returns (uint256 v) {
        val = v;
        return;
    } catch Error(string memory reason) {
        ...
        // revert(reason);
    } catch Panic(uint256 errorCode) {
        ...

    } catch (bytes memory lowLevelData) {
        ...
    }
}
```

Try / Catch with Custom Error

```
function doStuff4(bool _fail) external {  
    try conB.stuff1(_fail) returns (uint256 v) {  
        val = v;  
        return;  
    } catch MyError(uint256 _errorCode, uint256 _balance) {  
        ...  
        // revert(reason);  
    } catch (bytes memory lowLevel) {  
        ...  
    }  
}
```

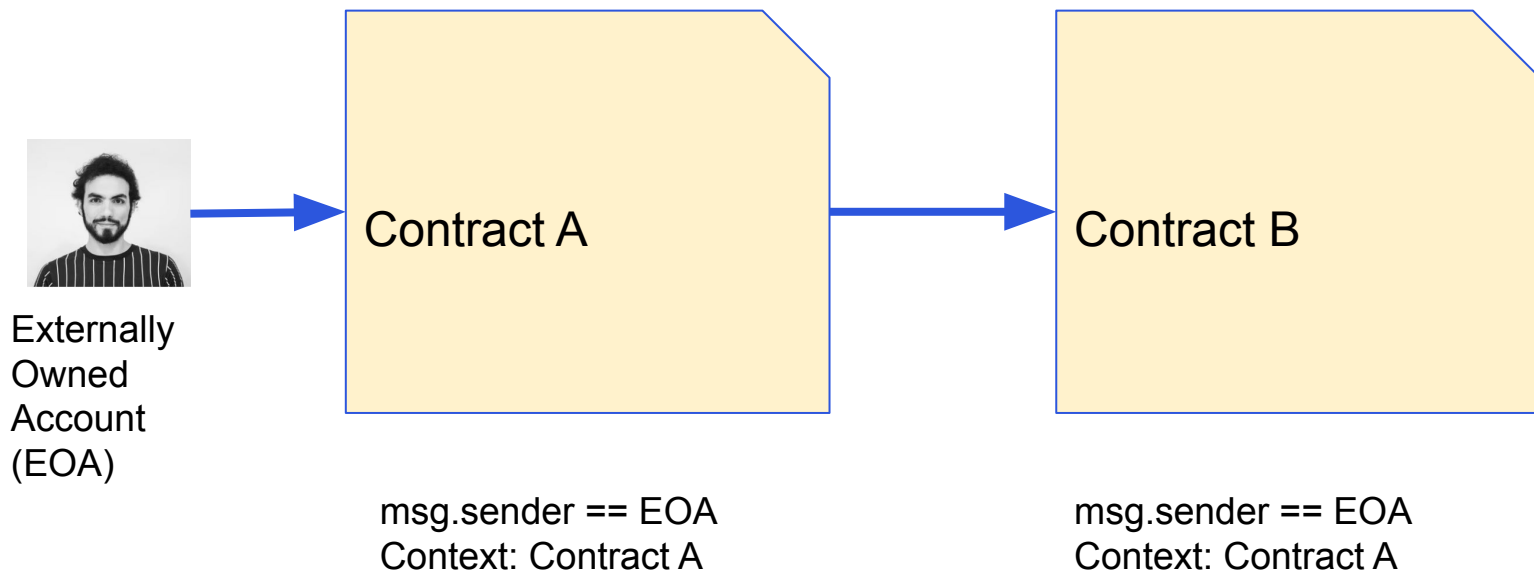
I couldn't get this to work - probably not supported yet.

OpenZeppelin Upgrade Proxy

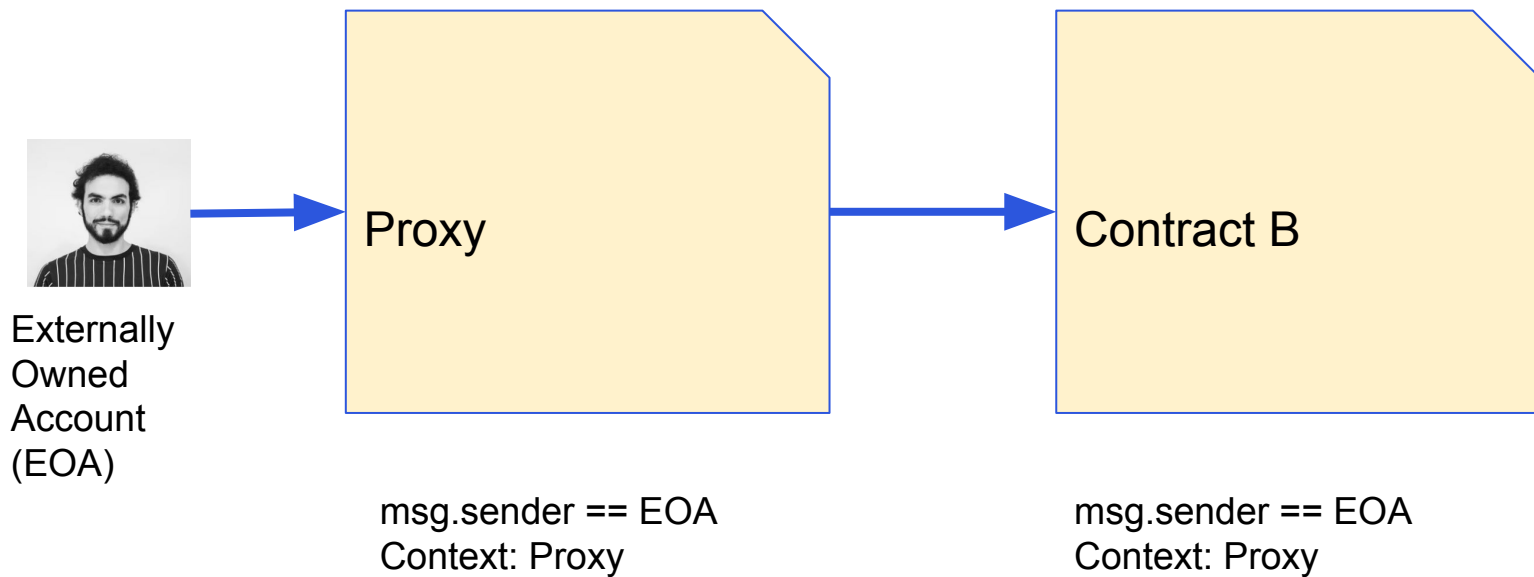
Proxy

- Open Zeppelin project have created an upgradable proxy contract.
- See this contract and the associated contracts:
<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/proxy/Proxy.sol>

Delegate Call

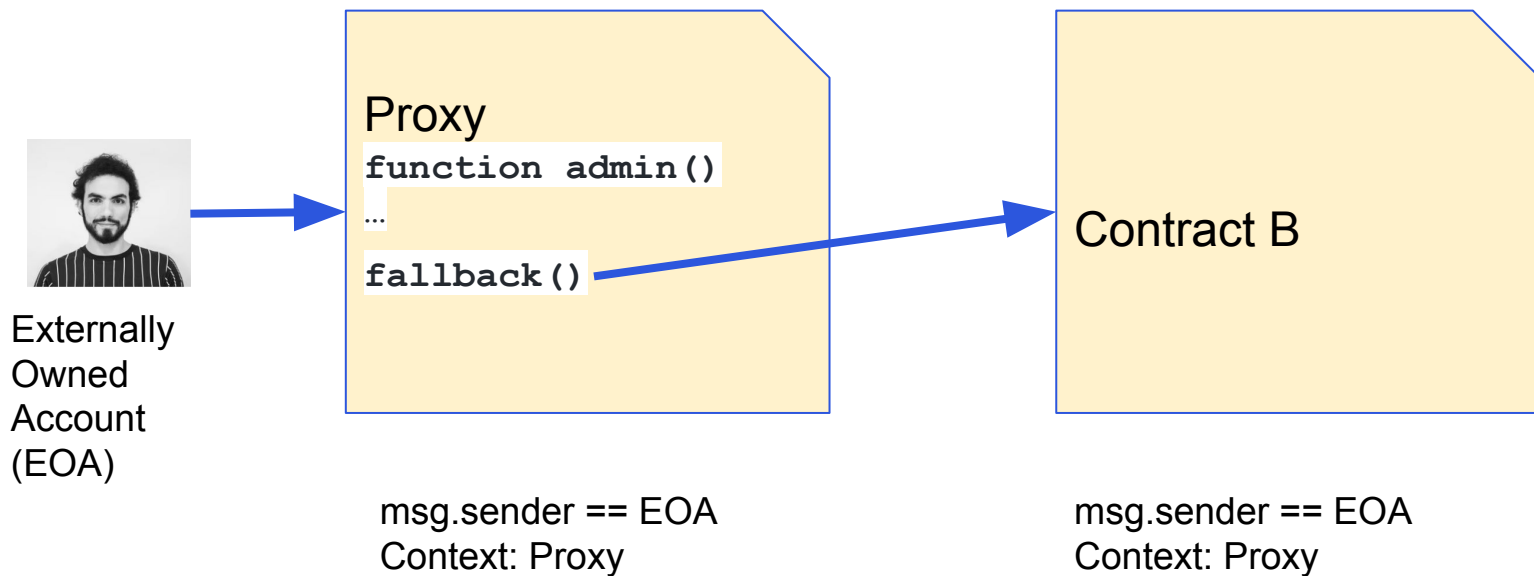


Delegate Call



- TransparentUpgradeableProxy* contract defines some external functions:
 - `function admin()`
 - `function implementation()`
 - `function changeAdmin(address newAdmin)`
 - `function upgradeTo(address newImplementation)`
 - `function upgradeToAndCall(address newImplementation, bytes calldata data)`
- Implementation contract can NOT have functions with the same function signatures.

Delegate Call



NOTE: This I have removed comments and changed the code slightly to make the code fit.

```
fallback() external payable virtual {
    address impl = getImplAddr();
    assembly {
        calldatacopy(0, 0, calldatasize())

        let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)

        returndatacopy(0, 0, returndatasize())

        switch result
        case 0 {
            revert(0, returndatasize())
        }
        default {
            return(0, returndatasize())
        }
    }
}
```

Proxy

Copy call data to memory, over-writing whatever is at memory location 0x00.

```
fallback() external payable virtual {  
    address impl = getImplAddr();  
    assembly {  
        calldatacopy(0, 0, calldatasize())  
  
        let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)  
  
        returndatacopy(0, 0, returndatasize())  
  
        switch result  
        case 0 {  
            revert(0, returndatasize())  
        }  
        default {  
            return(0, returndatasize())  
        }  
    }  
}
```

Proxy

Call the implementation, specifying the memory location and the size of the call data.

```
fallback() external payable virtual {
    address impl = getImplAddr();
    assembly {
        calldatacopy(0, 0, calldatasize())

        let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)

        returndatacopy(0, 0, returndatasize())

        switch result
        case 0 {
            revert(0, returndatasize())
        }
        default {
            return(0, returndatasize())
        }
    }
}
```

Proxy

Copy the return data to memory

```
fallback() external payable virtual {
    address impl = getImplAddr();
    assembly {
        calldatacopy(0, 0, calldatasize())

        let result := delegatecall(gas, impl, 0, calldatasize(), 0, 0)

        returndatacopy(0, 0, returndatasize())

        switch result
        case 0 {
            revert(0, returndatasize())
        }
        default {
            return(0, returndatasize())
        }
    }
}
```


Proxy

```
fallback() external payable virtual {
    address impl = getImplAddr();
    assembly {
        calldatacopy(0, 0, calldatasize())

        let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)

        returndatacopy(0, 0, returndatasize())

        switch result
        case 0 {
            revert(0, returndatasize())
        }
        default {
            return(0, returndatasize())
        }
    }
}
```

Return the information at memory location 0, the length of the return data, in a revert of return, depending on whether delegatecall reverted or not.

Memory Layout

Memory Slot	Proxy	Contract B
0		bVal
1		addr
2		balances
3		
0x360894a13ba1a32106 67c828492db98dca3e20 76cc3735a920a3ca505d 382bbc	Implementation	

Memory Layout

Memory Slot	Proxy	Contract B v1	Contract B v2
0		bVal	bVal
1		addr	addr
2		balances	balances
3			pause
0x360894a13ba1a 3210667c828492d b98dca3e2076cc3 735a920a3ca505d 382bbc	Implementation		

What about more complex code?

Version 1

```
contract ContractC {
    uint256 public val1;
    uint256 public val2;
    constructor () {
        val1 = 11;
        val2 = 12;
    }
}

contract ContractD {
    uint256 public val4;
    uint256 public val5;
    constructor () {
        val4 = 21;
        val5 = 22;
    }
}
```

```
contract ContractE is ContractC, ContractD {
    uint256 public val7;
    uint256 public val8;

    constructor () {
        val7 = 31;
        val8 = 32;
    }
}
```

Memory Slot	Proxy	Contract E v1
0		val1
1		val2
2		val4
3		val5
4		val7
5		val8
6		
0x360894a13b...	Implementation	

What about more complex code?

Version 1

```
contract ContractC {  
    uint256 public val1;  
    uint256 public val2;  
    constructor () {  
        val1 = 11;  
        val2 = 12;  
        val3 = 13;  
    }  
}
```

```
contract ContractD {  
    uint256 public val4;  
    uint256 public val5;  
    constructor () {  
        val4 = 21;  
        val5 = 22;  
    }  
}
```

```
contract ContractE is ContractC, ContractD {  
    uint256 public val7;  
    uint256 public val8;  
  
    constructor () {  
        val7 = 31;  
        val8 = 32;  
    }  
}
```

Memory Slot	Proxy	Contract E v1	Contract E v2
0		val1	val1
1		val2	val2
2		val4	val3
3		val5	val4
4		val7	val5
5		val8	val7
6			val8
0x360894a13b...	Implementation		

What about more complex code?

Revised Version 1

```
contract ContractC {  
    uint256 public val1;  
    uint256 public val2;  
    uint256 private dummy0;  
    constructor () {  
        val1 = 11;  
        val2 = 12;  
    }  
}
```

```
contract ContractD {  
    uint256 public val4;  
    uint256 public val5;  
    uint256 private dummy1;  
    constructor () {  
        val4 = 21;  
        val5 = 22;  
    }  
}
```

```
contract ContractE is ContractC, ContractD {  
    uint256 public val7;  
    uint256 public val8;  
  
    constructor () {  
        val7 = 31;  
        val8 = 32;  
    }  
}
```


Memory Slot	Proxy	Contract E v1
0		val1
1		val2
2		dummy0
3		val4
4		val5
5		dummy1
6		val7
7		val8
0x360894a13b...	Implementation	

What about more complex code?

Revised Version 2

```
contract ContractC {  
  uint256 public val1;  
  uint256 public val2;  
  uint256 public val3;  
  constructor () {  
    val1 = 11;  
    val2 = 12;  
    val3 = 13;  
  }  
}  
  
contract ContractD {  
  uint256 public val4;  
  uint256 public val5;  
  uint256 private dummy1;  
  constructor () {  
    val4 = 21;  
    val5 = 22;  
  }  
}
```

```
contract ContractE is ContractC, ContractD {  
  uint256 public val7;  
  uint256 public val8;  
  
  constructor () {  
    val7 = 31;  
    val8 = 32;  
  }  
}
```

Memory Slot	Proxy	Contract E v1	Contract E v2
0		val1	val1
1		val2	val2
2		dummy0	val3
3		val4	val4
4		val5	val5
5		dummy1	dummy1
6		val7	val7
7		val8	val8
0x360894a13b...	Implementation		

Upgradable Contracts

Good idea?

Bad idea?

Upgradable Contracts

- You have to trust the “owner” of an upgradable contract to not “bait and switch” contract logic.
- Note that “owner” could be a multi-sig wallet, or could be a single EOA.

Hidden Parameters / Hidden Context

Hidden Parameters

- Function signature:

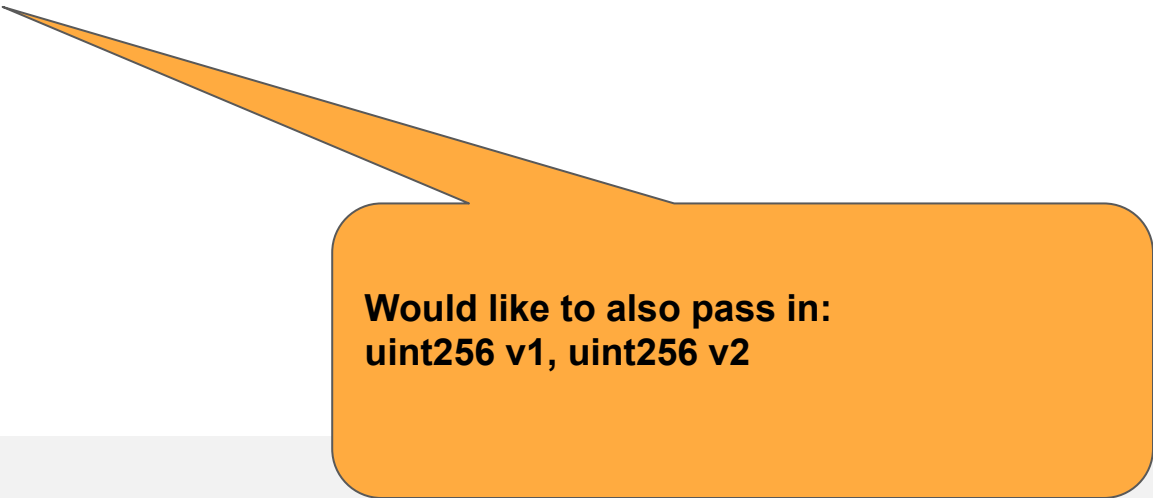
```
function transfer(address recipient, uint256 amount) external returns (bool);
```

- This could be defined by an interface.
- Sometimes, it would be good to be able to pass in some additional parameters.

Hidden Params

```
contract Source {  
    function callFunc(uint256 _val) external {  
        dest.func(11);  
    }  
}
```

```
contract Dest {  
    function func(uint256 _val) external {  
        ...  
    }  
}
```



Would like to also pass in:
uint256 v1, uint256 v2

Hidden Params

Call data:

Normal function call

bytes4	bytes
--------	-------

Function
Selector

ABI encoded parameters

Function call with hidden parameters

bytes4	bytes	bytes
--------	-------	-------

Function
Selector

ABI encoded parameters

ABI encoded hidden parameters

Hidden Params

Create the function selector and parameters.

```
contract Source {  
    function callFunc(uint256 _val) external {  
        bytes memory funcCall = abi.encodeWithSelector(dest.funcOneParam.selector, 11)  
        bytes memory authVals = abi.encodePacked(v1, v2),  
        bytes memory functionCallWithAuth = bytes.concat(funcCall, authVals);  
  
        bool isSuccess;  
        bytes memory returnValueEncoded;  
        (isSuccess, returnValueEncoded) = address(dest).call(functionCallWithAuth);  
  
        if (!isSuccess) {  
            revert(getRevertMsg(returnValueEncoded));  
        }  
    }  
}  
  
contract Dest {  
    function func(uint256 _val) external {  
        ...  
    }  
}
```

Hidden Params

Encode hidden parameters.

```
contract Source {
    function callFunc(uint256 _val) external {
        bytes memory funcCall = abi.encodeWithSelector(dest.funcOneParam.selector, 11);
        bytes memory authVals = abi.encodePacked(v1, v2);
        bytes memory functionCallWithAuth = bytes.concat(funcCall, authVals);

        bool isSuccess;
        bytes memory returnValueEncoded;
        (isSuccess, returnValueEncoded) = address(dest).call(functionCallWithAuth);

        if (!isSuccess) {
            revert(getRevertMsg(returnValueEncoded));
        }
    }
}

contract Dest {
    function func(uint256 _val) external {
        ...
    }
}
```

Hidden Params

Combine the function call and hidden parameters

```
contract Source {
    function callFunc(uint256 _val) external {
        bytes memory funcCall = abi.encodeWithSelector(dest.funcOneParam.selector, 11)
        bytes memory authVals = abi.encodePacked(v1, v2);
        bytes memory functionCallWithAuth = bytes.concat(funcCall, authVals);

        bool isSuccess;
        bytes memory returnValueEncoded;
        (isSuccess, returnValueEncoded) = address(dest).call(functionCallWithAuth);

        if (!isSuccess) {
            revert(getRevertMsg(returnValueEncoded));
        }
    }
}

contract Dest {
    function func(uint256 _val) external {
        ...
    }
}
```

Hidden Params

Do the function call and process reverts.

```
contract Source {
    function callFunc(uint256 _val) external {
        bytes memory funcCall = abi.encodeWithSelector(dest.funcParam.selector, 11)
        bytes memory authVals = abi.encodePacked(v1, v2);
        bytes memory functionCallWithAuth = bytes.concat(funcCall, authVals);

        bool isSuccess;
        bytes memory returnValueEncoded;
        (isSuccess, returnValueEncoded) = address(dest).call(functionCallWithAuth);

        if (!isSuccess) {
            revert(getRevertMsg(returnValueEncoded));
        }
    }
}

contract Dest {
    function func(uint256 _val) external {
        ...
    }
}
```

Hidden Params

Copy the variables from the end of the call data.

```
contract Dest {  
  function func(uint256 _val) external {  
    uint256 v1;  
    uint256 v2;  
    bytes calldata allParams = msg.data;  
    uint256 len = allParams.length;  
  
    assembly {  
      calldatacopy(0x0, sub(len, 64), 32)  
      v1 := mload(0)  
      calldatacopy(0x0, sub(len, 32), 32)  
      v2 := mload(0)  
    }  
    ...  
  }  
}
```

Note that you could check that the length of call data is long enough to contain the hidden parameters prior to executing this code.

Hidden Params

My use-case:

- In GPACK crosschain system, hidden parameters are used to add crosschain authentication parameters.
- Users program against a contract's API.
- Crosschain control contract injects the authentication parameters prior to calling the contract.

It is much cheaper to store information in call data than to temporarily store information in storage.

Questions



You Tube, Slack, Meet-up, Example Code

YouTube: <https://www.youtube.com/c/ethereumengineeringgroup>

Slack invitation link:

https://join.slack.com/t/eth-eng-group/shared_invite/zt-48ggg3kk-bUT3PWRn16hCpFclbcZrvQ

Meet-up: <https://www.meetup.com/ethereum-engineering/>

Example code: <https://github.com/drinkcoffee/EthEngGroupSolidityExamples>

Future Talks & Events

January 27 (Thursday): Eth2 Staking as a Service

- Lecky Lao, 12:30 pm Brisbane

February 9: Formal Verification of the QBFT Consensus Protocol

- Roberto Saltini, 11:00 am Brisbane

February 23: TBA

- TBA

March 23: 100th Meet-up Talk - a Surprise

- TBA

April or May: Ethereum Gas Usage & Gas Optimisation

- Peter Robinson